Boogle Dox — 3–5 Minute Walkthrough Script

Use this as an outline, not as a claim that the feature already works. Remove or adapt sections for any unfinished functionality.

0:00–0:30 — Intro

"Hi, this is Boogle Dox, my take on a lightweight browser-based document editor. I focused on making document creation, editing, saving, importing, and sharing feel straightforward. I intentionally scoped it as a usable product slice rather than trying to recreate all of Google Docs."

0:30–1:40 — Main editing flow

[Show the dashboard.] "I can see the documents I own separately from documents shared with me. I'll create a document, give it a name, add some text, and try the formatting toolbar. The menus keep familiar labels like File, Edit, View, Insert, Format, and Tools, but I only included actions I could make work reliably."

[Demonstrate working formatting, undo/redo, and save.] "Now I'll refresh and reopen it to show that the content and formatting persist."

1:40–2:20 — Import

"I also included a file import flow so users can bring in existing text. For this version I limited it to TXT and Markdown instead of building a complex DOCX importer. The supported formats are stated in the interface."

[Import a supported file and, if possible, demonstrate an unsupported-file error.]

2:20–3:20 — Sharing and authorization

"Sharing is important because a copied document link shouldn't automatically mean access. As the owner, I can grant a second account viewer or editor permissions. I'll copy the link and open it as the second user. A viewer can read but shouldn't be able to change anything."

[Show two accounts, and ideally a denied attempt from an unauthorized account.]

3:20–4:10 — Architecture and tradeoffs

"I used a React-based browser app and a JavaScript/TypeScript server stack so I could keep the experience interactive without switching languages. The editor stores structured content, while the database stores ownership and sharing grants. The server enforces permissions instead of relying only on what the UI shows."

"I deprioritized simultaneous multi-cursor editing. My longer-term idea is an edit-session model that tells people when someone else is editing, so they can work against a saved version without conflicts."

4:10–4:50 — AI and wrap-up

"I used [actual tools] to help with [actual tasks]. I personally made the scope and interface decisions, reviewed [specific code], and verified [specific tests]. If I had another two to four hours, I would focus on [actual next priority]."

[Close with the working URL and a quick reminder of known limitations.]